

MULTILIZERTM

G L O B A L L O C A L I Z A T I O N



VCL Tutorial

Using MULTILIZER™ VCL Edition 4.2

September 1999

Copyright © 1999 Innoview Data Technologies, Ltd. All rights reserved.

MULTILIZER is a trademark of Innoview Data Technologies, Ltd.

Delphi is a registered trademark of Inprise Corporation.

C++Builder is a trademark of Inprise Corporation.

Table of Contents

1	Preface	1
2	Introduction to the tutorial	2
3	Opening the Monolingual Application	3
4	Making the Application Multilingual	4
5	Creating a Dictionary for the Application	6
6	Internationalizing the Code.....	12
7	Changing Language On Run Time	19
8	Adding Western Languages	22
9	Adding Custom Locales	23
10	Adding Non-Western Languages	25
11	Adding Far Eastern Languages.....	28
12	Adding Middle Eastern Languages.....	29

1

Preface

The purpose of this tutorial is to familiarize you with common software localization tasks when using MULTILIZER™. To obtain the most precise definitions on component use and technical details, please refer to the on-line help.

The following items can be found in **intro.pdf**

- Typographical conventions used in this document.
- General information on localization.
- How does MULTILIZER™ work, what is Language Manager for?
- What is the "Native" language used in Language Manager?

The following items can be found in **langman.pdf**

- Language Manager related tasks.

2

Introduction to the tutorial

In this tutorial we are going to create a multilingual application. The application will be a simple driving-time calculator, Dcalc which a user can use to calculate the average driving time for a given distance.

The Dcalc application is very simple but still it uses most of the features of MULTILIZER. The creation of the application is divided into ten lessons, each covering one or more MULTILIZER functions.

This tutorial is written for Delphi 5, and C++Builder 4. Using an older version is almost identical. Some of the menu commands may vary.



All screen-captured images have been taken when the active language of Language Manager is English. Set English on by clicking the Earth-image on the left side of the Language Manager's tool bar.

The Dcalc sample application locates in the `<compiler>\Samples\Dcalc` subdirectory of your MULTILIZER directory. `<compiler>` is *Delphi1*, *Delphi2*, *Delphi3*, *Delphi4*, *Delphi5*, *CBuilder1*, *CBuilder3* or *CBuilder4* depending on your compiler version.

To see more about how to use MULTILIZER read the online help and study the other sample applications found in the `<compiler>\Samples` subdirectory.

The MULTILIZER setup created the following program group:



Figure 1. Multilizer Program Group.

Before you can start building Dcalc you have to install MULTILIZER components in the Component Palette. To get the information on how to install them, see the readme files. To open the readme double click the *XXX Readme* icon of the compiler, where XXX is your compiler (e.g. Delphi 5).

3

Opening the Monolingual Application

We could start from scratch but in most cases it is a completed application or at least an application under construction that you want to globalize. This is what we are going to do. The <compiler>\Samples\Tutorial contains the English Dcalc. Open it, compile it, and finally run it.

The application should look like this:

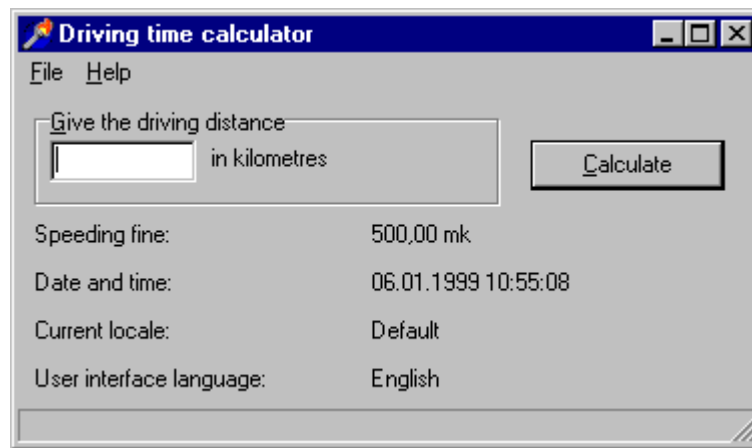


Figure 2. The monolingual application using English.

The user interface language is English and the application uses the default locale, which in this case is Finnish (Finland). The speeding ticket is formatted using the Finnish currency format (marks) and the date and time is also formatted using the Finnish format. Standard VCL provides this kind of localization through its locale variables.

In the following chapters we will make Dcalc truly multilingual step-by-step.

4

Making the Application Multilingual

The first step is to make Dcalc multilingual, by just dropping two components on the form. Select the MULTILIZER sheet from the Component Palette. Drop the TlvTranslator on the form. Drop the TlvTestDictionary on the form.

The result should look like this:

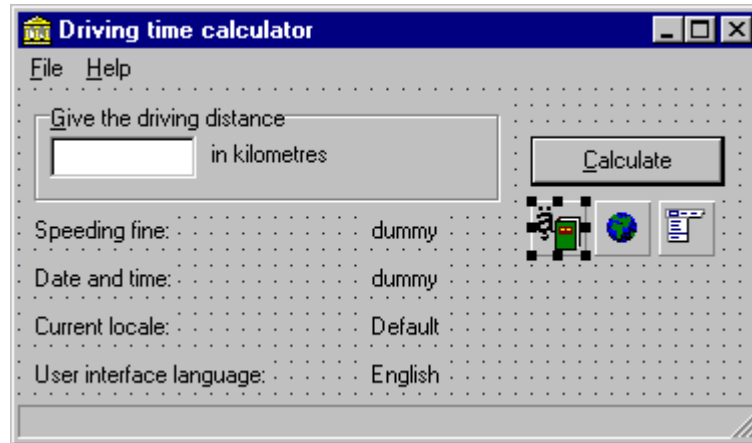


Figure 3. The translator and dictionary components have been added on the form.

What are these two components for? TlvTestDictionary is one of the dictionary components of MULTILIZER. A dictionary component provides string or phrase translation for the application. Normally each application contains one dictionary component that contains all the translation data of the application. MULTILIZER contains several different dictionary components: one for getting the translation data from a text file, another for data from a database, etc.

The TlvTestDictionary is a special case. It does not require any dictionary data but it makes the translation on the fly by changing the original string to a test string. In a normal case you can not use the test dictionary in your final application because the translation is not a real language. However, the test dictionary is really handy in the development phase.

The TlvTranslator on the other hand is the component that does all the work. It scans the form before it becomes visible and translates the user interface string from the original value to the current language.

Compile and run Dcalc. It should look like this:

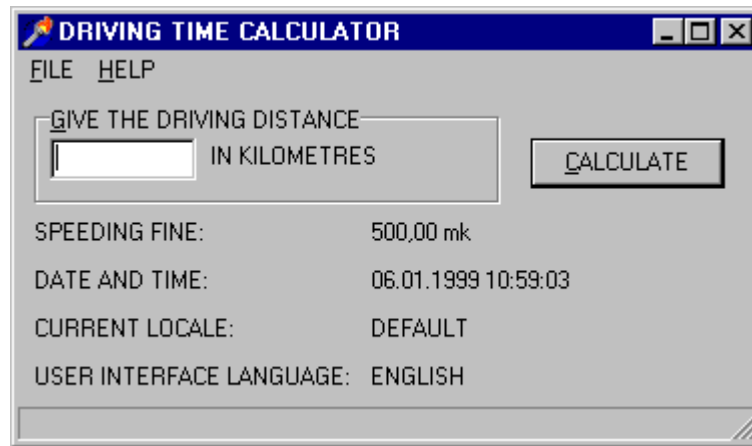


Figure 4. “Translated” application. The test dictionary translated every string to an upper case string.

As you can see, every user interface string is now in upper case. The translator changed every string type property after the form had been loaded from the resource. By default the test dictionary translates every string by putting it in upper case.



NOTE!

Generally you need to add one Dictionary component on the application main form and one Translator component on every form in the application. Check the on-line help topic “Translator Usage” for more information on this.



MORE INFO

For additional information on using the test dictionary, see the online help topic “TivTestDictionary”.

This was a quick demonstration of the power of MULTILIZER. In the next chapter we will create a real dictionary that contains real languages.

5

Creating a Dictionary for the Application

Double click the Language Manager icon from the MULTILIZER program group to start Language Manager.

Choose File | New to start the Dictionary Wizard. The following dialog box appears:

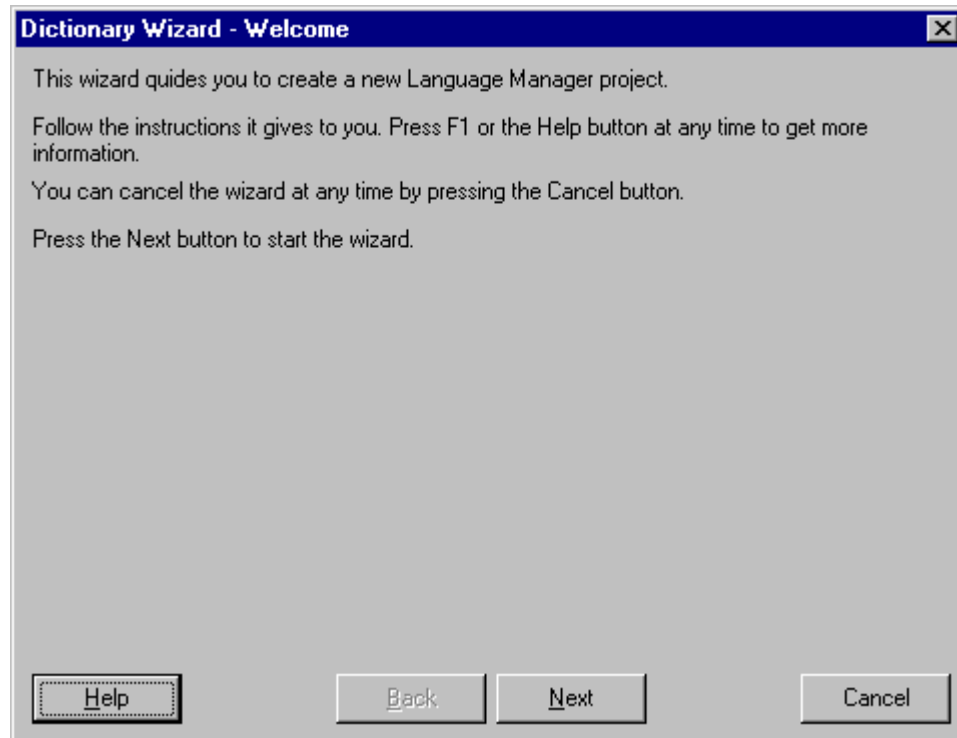


Figure 5. The Welcome sheet.

Press the Next button. The Source Directory sheet appears.

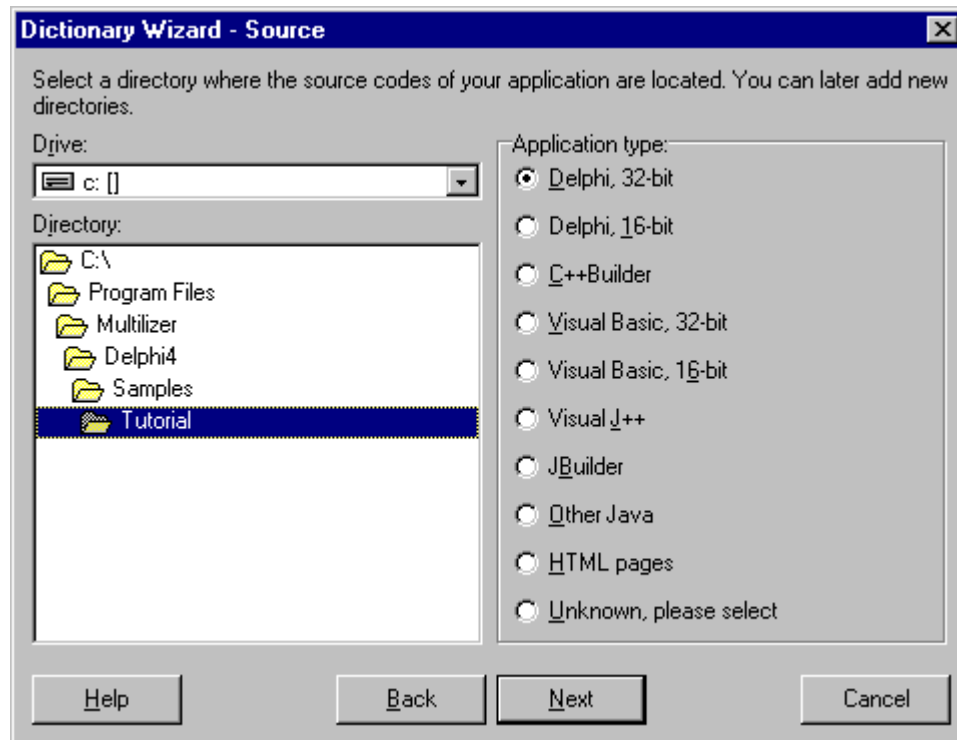


Figure 6. The Source sheet is used to enter the source directory.

This sheet specifies the directory where your application is located. Choose the <compiler>\Samples\Tutorial subdirectory of your MULTILIZER setup. If your application locates on multiple directories you can later add more directories. Dictionary Wizard detects the application type. If it is wrong you can select the right type.

Press the Next button. The Project Information sheet appears:

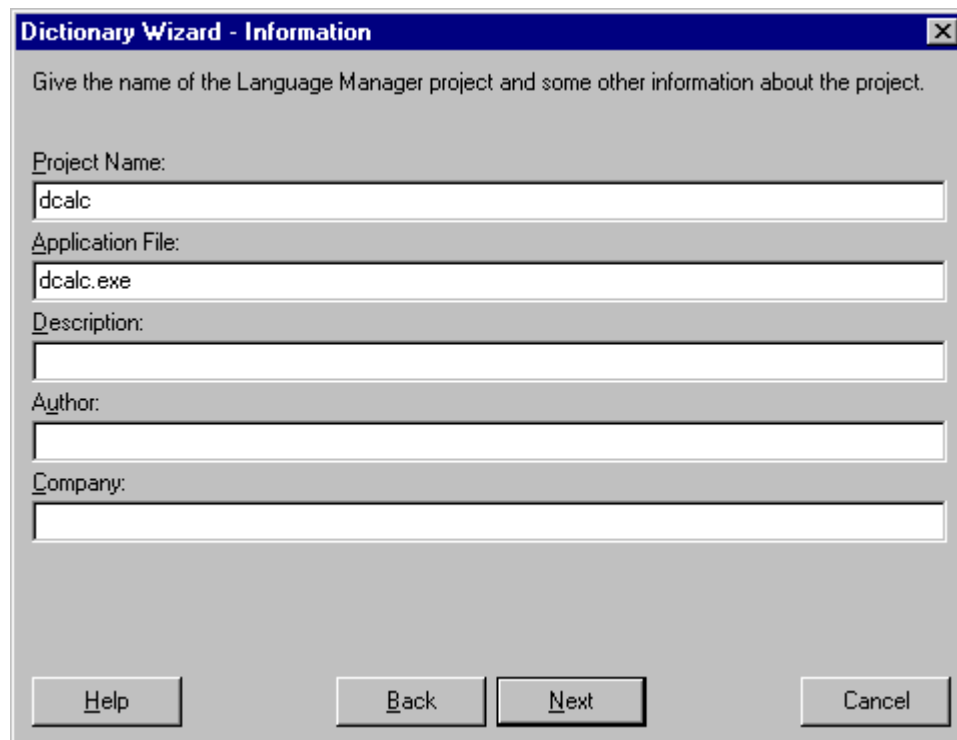


Figure 7. The Information sheet is used to enter the project name and application file.

This sheet specifies the directory name and the application's EXE file. You can also enter other information about the dictionary, the author, and the company.

Press the Next button. The Dictionary Type sheet appears:

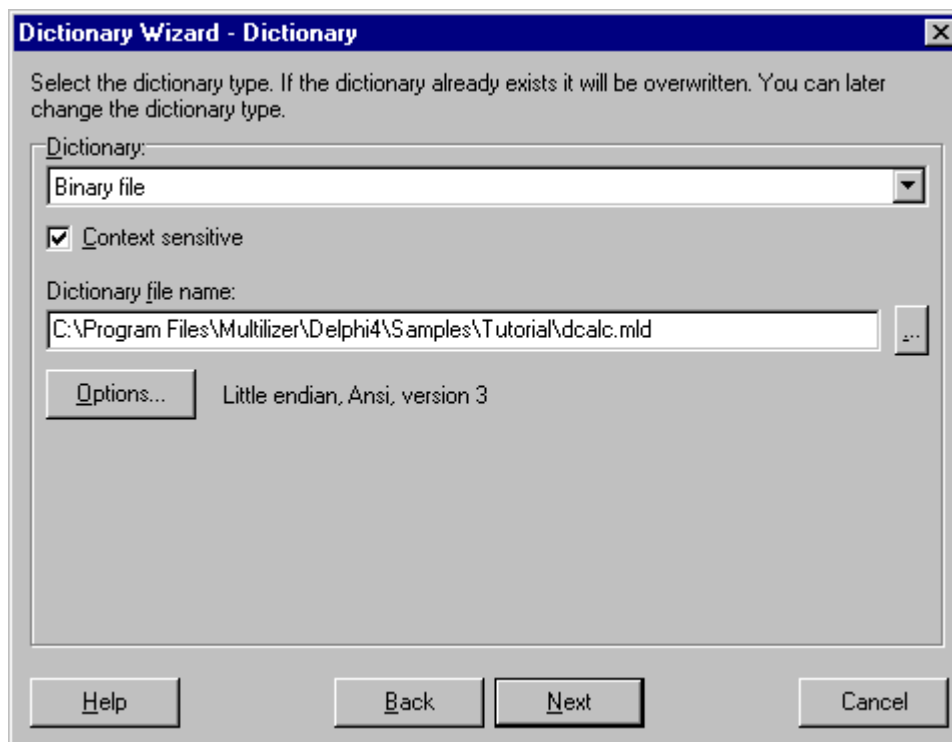


Figure 8. The Dictionary sheet is used to specify the dictionary type.

This sheet specifies the type of dictionary. The online help describes each dictionary type. Accept the default dictionary (Binary file) by pressing the **Next** button.

The Languages sheet appears:

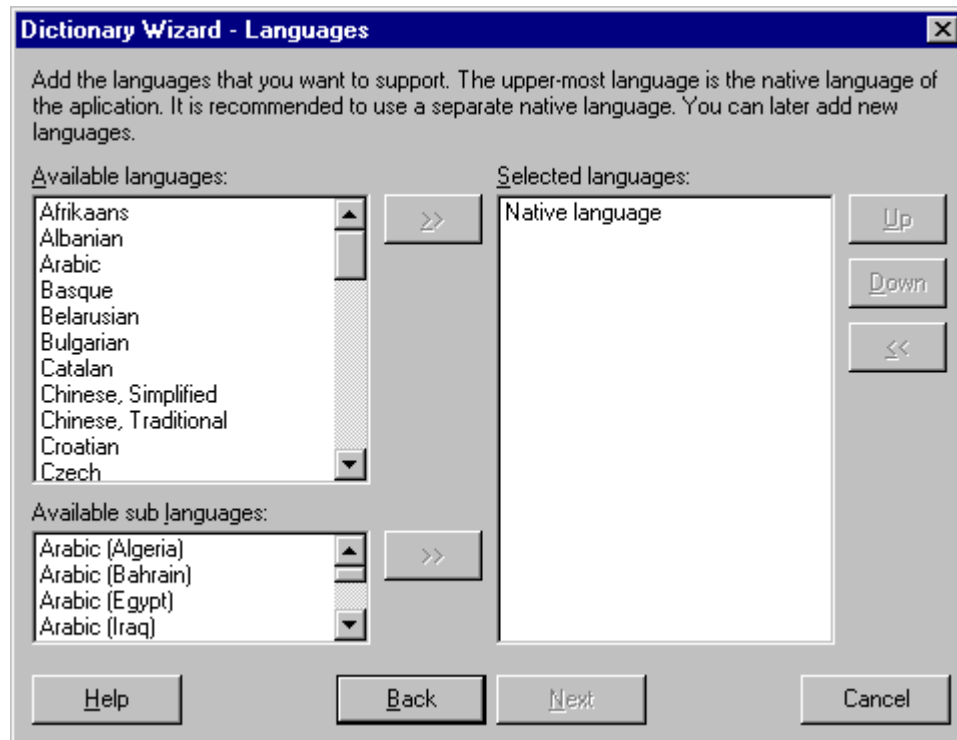


Figure 9. The Languages sheet is used to add languages to the dictionary.

This sheet lets you add string languages to the dictionary. By default the dictionary contains the native language. This is the language you used to program your application. From Available languages select English and press the >> button. This adds English support to the dictionary.

Add some other European language (we will take care of Far Eastern and Middle Eastern languages later). If you add Finnish the result should look like this:

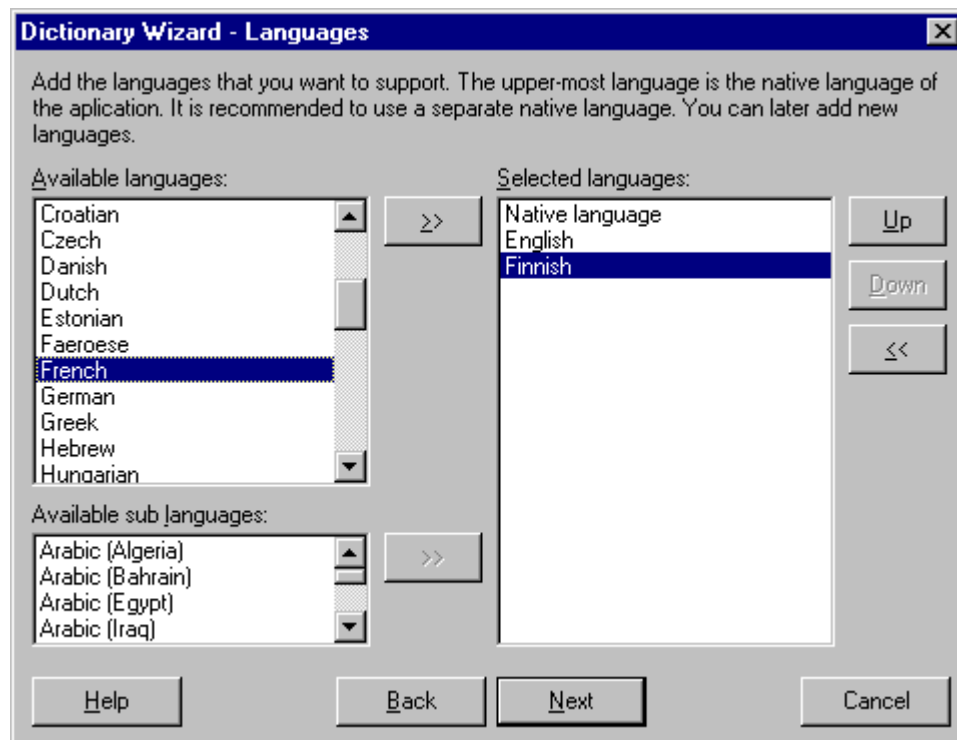
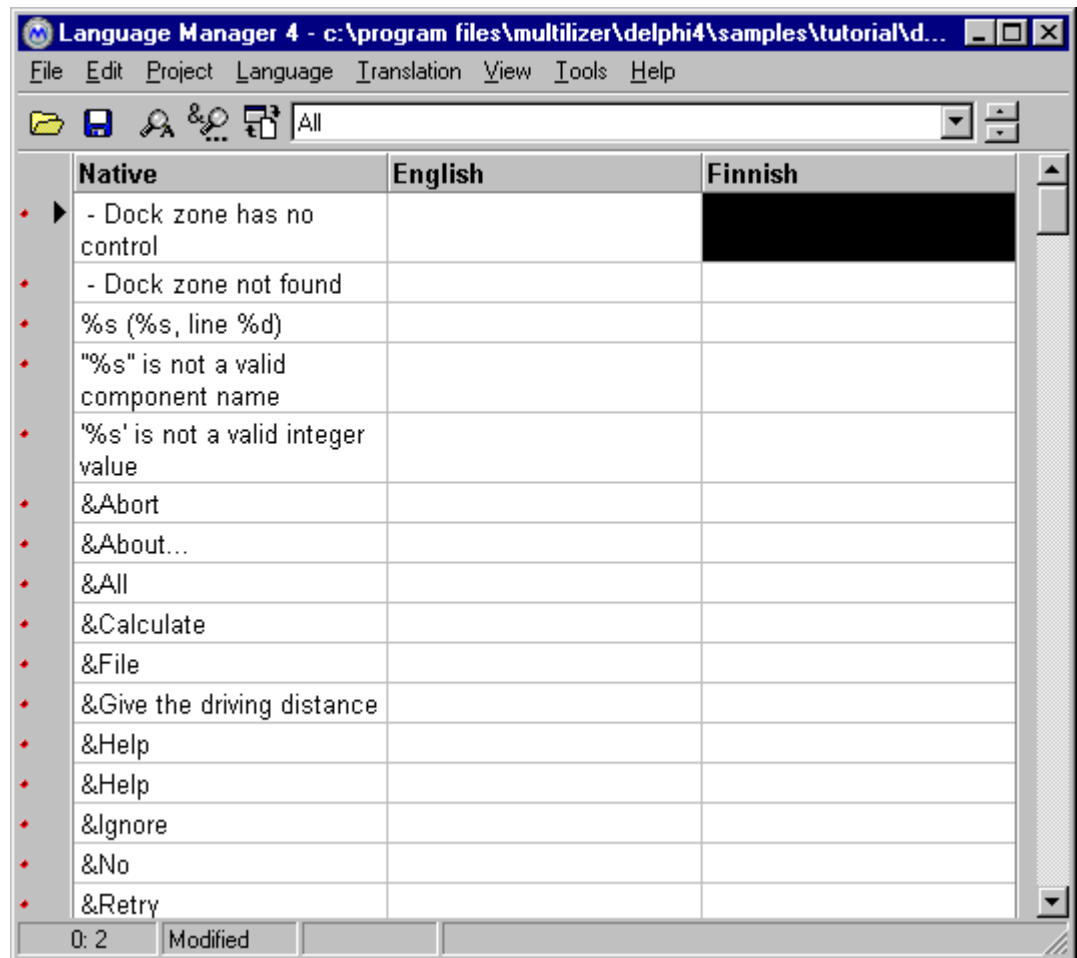


Figure 10. Native, English and Finnish added to dictionary.

Press the **Next** button. The Ready to create dictionary sheet appears. Now you have almost finished creating the dictionary.

Press the **Finish** button to end Dictionary Wizard. The following dictionary grid appears.



Native	English	Finnish
• ▶ - Dock zone has no control		
• - Dock zone not found		
• %s (%s, line %d)		
• "%s" is not a valid component name		
• '%s' is not a valid integer value		
• &Abort		
• &About...		
• &All		
• &Calculate		
• &File		
• &Give the driving distance		
• &Help		
• &Help		
• &Ignore		
• &No		
• &Retry		

Figure 11. The dictionary grid.

Save the dictionary by choosing File | Save As.

We could translate the Finnish (or your own language) column manually by entering the translations. However there is an easier way: using the glossaries. They implement the translation memory. The glossary files contain the translations of the most common strings or phrases (e.g. File, Open, About, etc). To make your translation job easier we can first let Language Manager use the glossaries to translate those strings.

Language Manager contains several glossary files for most supported languages. You can edit the glossary files list by choosing Tools | Glossaries.



For additional information on glossaries, find the online help topic "Glossaries".

Open Dcalc's dictionary by choosing the uppermost item from the File | Reopen menu.

The next task is to translate the Finnish (or your own language) column. We can now use the master dictionary to translate some of the strings. Right mouse click the header of the Finnish column. A popup menu appears. Choose Translate | Using Glossaries. Language Manager translates most of the strings. It is up to you to translate the rest.

Use the arrow keys to move to the right cell and start typing. Translate every row. Because the native language is English you do not have to translate the English column. The dictionary uses the native string if the translation is not found.



For additional information on coping with failing translations, find the online help topic "MissingTranslation".

6

Internationalizing the Code

We have the dictionary file now. Let's use it. Delete the TlvTestDictionary component from the form. Add the TlvBinaryDictionary component. Choose the component and move to the Object Inspector. Set the FileName property to `dcalc.mld` which you created in previous lesson.



If you pressed the ... button and browsed the file name, the Object Inspector would add the full file name (e.g. `C:\Program Files\Multilizer\Delphi5\Samples\tutorial\dcalc.mld`). This may cause troubles when deploying your application so remove the path part from the file name.

Later you will learn how to include the dictionary file in the EXE file so that there is no need to deploy it. The Object Inspector should look like this:

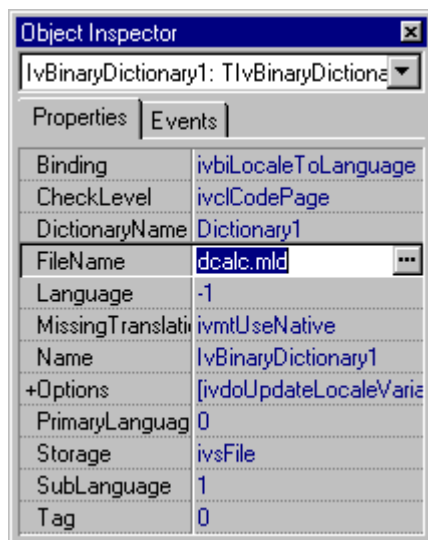


Figure 12. The Object Inspector showing the properties of a binary dictionary component.

Let's study some of the properties. The Language property specifies the active language. It is `-1`. This makes MULTILIZER check the current locale of the user and find the language that matches the locale. If none is found the first (non-native) language is used.

The PrimaryLanguage and the SubLanguage properties specify the active locale. The active language determines the language of the user interface. The active locale, however, determines the locale used by the application. The locale is a country and language specific object that controls how the date, time, currency, number, etc. are formatted.

The PrimaryLanguage property is `0` and the SubLanguage property is `1`. These make MULTILIZER use the default locale of the user. See more in the online documentation under the TlvDictionary topic.

In our case the dictionary contains English and Finnish. If the locale setting of the user is Finnish (Finland) the user interface of Dcalc will be in Finnish and the locale will be Finnish (Finland).



One important property is Storage. It specifies how the dictionary is used on run-time. In this example, the dictionary will be in an external file also on run-time. The dictionary could also be embedded in the exe or stored in resource data.

Run the application. It should look like this:

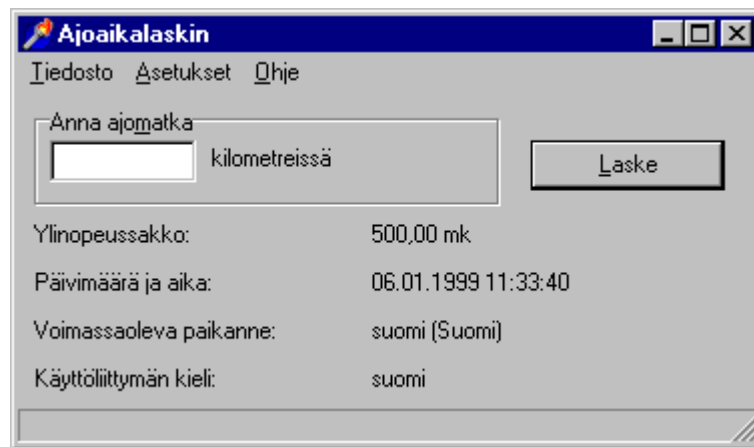


Figure 13. Dcalc in Finnish.

Making a multilingual application is this simple. In a simple case, this is all you have to do to make a multilingual application. In most other cases you have to do a little bit more.

If the program contains items which are just country (locale)-specific and hard coded in the source, they must be removed. This phase is called internationalization: it makes your software international and language/country independent. The next phase would then be to localize the program, i.e., add for each target country the locale-specific issues. This is done easily by using MULTILIZER. The remaining document discusses how to do this.

Dcalc calculates the average driving time. Most countries use the metric system, where the distance is expressed in kilometers. However in the US miles are used. Let's study how to make Dcalc compatible with both kilometers and miles.

When pressing the Calculate button Dcalc calls the following event:

Delphi

```

procedure TMainForm.CalculateButtonClick(Sender: TObject);
var
    distance, hours, minutes: Integer;
begin
    try
        distance := StrToInt(DistanceEdit.Text);
        if distance < 0 then
            raise Exception.Create('Negative');

        hours := distance div 100;
        minutes := Round(0.6*(distance mod 100));

        MessageDlg(
            Format(
                'The average driving time is %0:d hours and %1:d minutes.',
                [hours, minutes]),
            mtInformation,
            [mbOK],
            0);
    except
        MessageDlg(
            Format(
                '"%s" is not a valid distance!',
                [DistanceEdit.Text]),
            mtError,
            [mbOK],
            0);
        DistanceEdit.SetFocus;
    end;
end;

```

C++Builder

```

void __fastcall TMainForm::CalculateButtonClick(TObject *Sender)
{
    try
    {
        if (DistanceEdit->Text.Length() == 0)
        {
            DistanceEdit->Text = " ";
            throw Exception("");
        }
        int distance = StrToInt(DistanceEdit->Text);
        if (distance < 0)
            throw Exception("");

        int hours = distance/100;
        int minutes = 0.6*(distance%100);

        MessageDlg(
            Format(
                "The average driving time is %d hours and %d minutes.",
                OPENARRAY(TVarRec, (hours, minutes))),
            mtInformation,
            TMsgDlgButtons() << mbOK,
            0);
    }
    catch (...)
    {
        MessageDlg(
            Format(
                "\"%s\" is not a valid distance!",
                OPENARRAY(TVarRec, (DistanceEdit->Text))),
            mtError,
            TMsgDlgButtons() << mbOK,
            0);
        DistanceEdit->SetFocus();
    }
}

```

When the English (United States) locale is on the user gives the distance in miles. To convert miles to kilometers add the following just before
hours := distance div 100;

Delphi

```
if IvBinaryDictionary1.LocaleData.MeasurementSystem = ivmsUS then  
    distance := Trunc(MILE_IN_METERS*distance/1000);
```

C++Builder

```
if (IvBinaryDictionary1->LocaleData->MeasurementSystem == ivmsUS)  
    distance = MILE_IN_METERS*distance/1000;
```

This is enough for the system to convert miles to kilometers but not for the user. He or she will most definitely be a bit confused if the user interface still prompts in kilometers. be “To make the user interface react to the locale change, add the OnLocaleChange event to the dictionary and write the following code:

Delphi

```

procedure TMainForm.IvBinaryDictionary1LocaleChange(Sender: TObject);
begin
    if IvBinaryDictionary1.LocaleData.MeasurementSystem = ivmsMetric then
    begin
        UnitLabel.Caption := Translate('in kilometres');
        DistanceEdit.Hint := Translate('Give the driving distance in
kilometres');
    end
    else
    begin
        UnitLabel.Caption := Translate('in miles');
        DistanceEdit.Hint := Translate('Give the driving distance in miles');
    end;

    SpeedingFine.Caption := Format('%m', [500.0]);
    CurrentTime.Caption := DateTimeToStr(Now);
    CurrentLocale.Caption :=
        IvBinaryDictionary1.LocaleData.GetDisplayName(
            ivdnTranslated,
            IvBinaryDictionary1);
    CurrentLanguage.Caption :=
        Translate(IvBinaryDictionary1.LanguageData.EnglishName);
end;

```

C++Builder

```

void __fastcall TMainForm::IvBinaryDictionary1LocaleChange (TObject
*Sender)
{
    if (IvBinaryDictionary1->LocaleData->MeasurementSystem == ivmsMetric)
    {
        UnitLabel->Caption = Translate("in kilometres");
        DistanceEdit->Hint = Translate("Give the driving distance in
kilometres");
    }
    else
    {
        UnitLabel->Caption = Translate("in miles");
        DistanceEdit->Hint = Translate("Give the driving distance in miles");
    }

    SpeedingFine->Caption = Format("%m", OPENARRAY(TVarRec, (500.0)));

    CurrentTime->Caption = DateTimeToStr(Now());
    CurrentLocale->Caption =
        IvBinaryDictionary1->LocaleData->GetDisplayName(
            ivdnTranslated,
            IvBinaryDictionary1);
    CurrentLanguage->Caption =
        Translate(IvBinaryDictionary1->LanguageData->EnglishName);
}

```

First the code checks the measurement system. This is done by comparing the global variable MeasurementSystem. The code updates the label and hint. Let's study the following code in more detail:

Delphi

```
UnitLabel.Caption := Translate('in kilometres');
```

C++Builder

```
UnitLabel->Caption = Translate("in kilometres");
```

In a monolingual application you would have used the following code:

Delphi UnitLabel.Caption := 'in kilometres';

C++Builder UnitLabel->Caption = "in kilometres";

This isn't the proper way in a multilingual application because the same EXE file must work on every language and locale. That's why the native string is translated before being assigned to the Caption property.

The lower part of the event updates the speeding fine, current time, active language name, and active locale name.

The monolingual Dcalc contains the following event:

Delphi

```
procedure TMainForm.FormActivate(Sender: TObject);
begin
    SpeedingFine.Caption := Format('%m', [500.0]);
    CurrentTime.Caption := DateTimeToStr(Now);
end;
```

C++Builder

```
void __fastcall TMainForm::FormActivate(TObject *Sender)
{
    SpeedingFine->Caption = Format("%m", OPENARRAY(TVarRec, (500.0)));
    CurrentTime->Caption = DateTimeToStr(Now());
}
```

The event is not required any more because the IvBinaryDictionary1LocaleChange event updates the labels. You can remove it.

We need to make a few modifications to the CalculateButtonClick event to make the message boxes multilingual. Consider the following code:

Delphi

```
MessageDlg(
    Format(
        'The average driving time is %0:d hours and %1:d minutes.',
        [hours, minutes]),
    mtInformation,
    [mbOK],
    0);
```

C++Builder

```
MessageDlg(
    Format(
        "The average driving time is %d hours and %d minutes.",
        OPENARRAY(TVarRec, (hours, minutes))),
    mtInformation,
    TMsgDlgButtons() << mbOK,
    0);
```

MULTILIZER translates the message dialogs but not the message text. You must use the Translate method to translate it.

Delphi

```

MessageDlg(
    Format(
        Translate('The average driving time is %0:d hours and %1:d
minutes.'),
        [hours, minutes]),
    mtInformation,
    [mbOK],
    0);

```

C++Builder

```

MessageDlg(
    Format(
        Translate("The average driving time is %d hours and %d minutes."),
        OPENARRAY(TVarRec, (hours, minutes))),
    mtInformation,
    TMsgDlgButtons() << mbOK,
    0);

```

Remember that you have to translate the exception message, as well.

Delphi

```

MessageDlg(
    Format(
        Translate('"%s" is not a valid distance!'),
        [DistanceEdit.Text]),
    mtError,
    [mbOK],
    0);

```

C++Builder

```

MessageDlg(
    Format(
        Translate("\'%s\'" is not a valid distance!"),
        OPENARRAY(TVarRec, (DistanceEdit->Text))),
    mtError,
    TMsgDlgButtons() << mbOK,
    0);

```

Translate the message box in the AboutMenuClick event.

Delphi

```

MessageDlg(
    Translate('Dcalc is a multilingual application that calculates the
average driving time'),
    mtCustom,
    [mbOK],
    0);

```

C++Builder

```

MessageDlg(
    Translate"Dcalc is a multilingual application that calculates the
average driving time"),
    mtCustom,
    TMsgDlgButtons() << mbOK,
    0);

```

Assign the `IvBinaryDictionary1LocaleChange` event to the `OnLanguageChange` event as well. This is because the above messages need to be updated every time either the active language or active locale changes.

Add `IvMlUtils` unit to the uses clause of the implementation part:

Delphi

```

uses
    IvMlUtil;

```

C++Builder

```

#pragma package(smart_init)
#pragma link "IvMlUtil"

```

7

Changing Language On Run Time

Dcalc now has the ability to adapt to the current language and locale settings of the user. What about changing the language and/or locale on run time? Is this possible? Yes. Double click the MainMenu1 component. The menu editor appears. Add a Language... menu item to the Language menu.

The result should look like this:

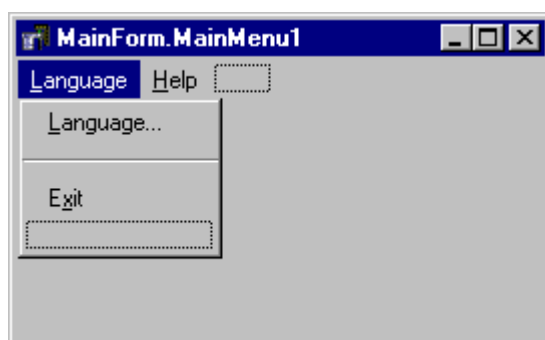


Figure 14. Add the Language menu item to the main menu.

Write the following event handler to the Language... menu item:

Delphi

```
procedure TMainForm.LanguageMenuClick(Sender: TObject);
var
    language: Integer;
begin
    if SelectLanguage(Self, IvBinaryDictionary1, '', [], 0, language) then
        IvBinaryDictionary1.Language := language;
end;
```

C++Builder

```
void __fastcall TMainForm::LanguageMenuClick(TObject *Sender)
{
    int language;

    if (SelectLanguage(this, IvBinaryDictionary1, "", options, 0,
        language))
        IvBinaryDictionary1->Language = language;
}
```

The SelectLanguage shows the Language Select dialog box of MULTILIZER. It contains a list of available languages that the user can select. After calling the SelectLanguage function the event sets the new active language by setting the Language property of the dictionary component.

Add the IvLanguD unit to the uses clause of the implementation part. This is because the SelectLanguage function locates on that unit.

Run the application and choose Language | Language... (Or Kieli | Kieli... if you have Finnish active). The following dialog box appears:

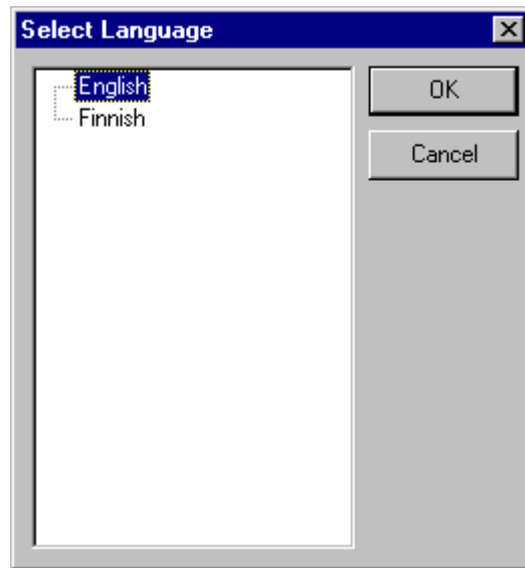


Figure 15. The *Select Language* dialog box lets the user select the active language on run time.

The `SelectLanguage` dialog box shows all available languages in a tree view. The available language does not necessarily mean all the languages in the dictionary. It may be that your operating system cannot cope with all character scripts.

For example you need to have a bi-directional OS to properly display Arabic or Hebrew. Also most Western OS versions lack support for Cyrillic or Far Eastern languages. You can make the dictionary component and `SelectLanguage` function display every language by setting the `CheckLevel` property of the dictionary component to `ivclNone`.

Select a new language and press the OK button. The active language (user interface) of Dcalc changes to that language. By default the active locale also changes to the default locale of the language. You can set the active language and locale independently by removing the `ivdoBindLocale` from the `Options` property of the dictionary component.

The `SelectLanguage` dialog box contains strings that need to be translated as well. Also the `IvMessageBox` uses several strings (e.g. "OK", "Cancel"). The master string table of Language Manager contains all these constant strings. All you need to do is to add them to your dictionary. Start Language Manager. Open the dictionary. Choose `Project | Include | System String`. The `System String` dialog box appears. Check `Language Dialog`, `Message Box`, and `System Menu` check boxes.

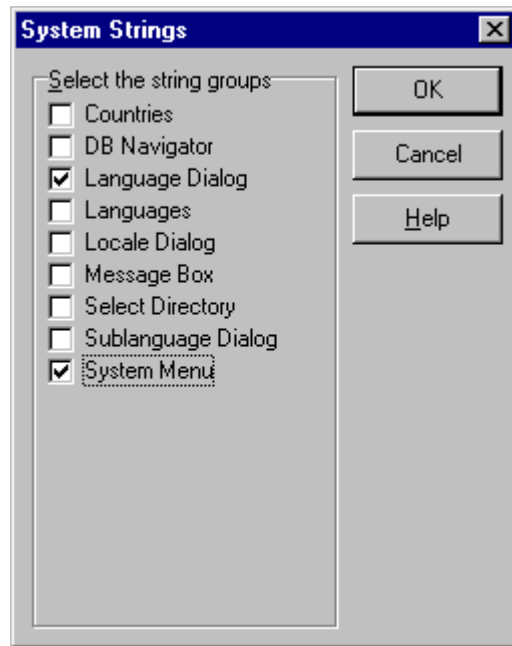


Figure 16. The System Strings dialog box lets the developer add strings used by the system or by the standard components.

Press the OK button. Language Manager adds the strings used by the Select Language dialog and the system menu. The glossaries contain translations of these strings. You do not have to translate them. Let Language Manager get the translations from the glossaries: Select any cell in the Finnish column. Choose Language | Translate | Using Glossaries.

Now our Dcalc application is fully multilingual. The user interface, locale settings, and input measures match the local settings. The user can even change the language on run time. The remaining chapters describe some of the advanced features of MULTILIZER.

8

Adding Western Languages

It is quite likely that you need to add new languages to your applications after the original languages. How to do that? This is the most powerful feature of MULTILIZER. After making your application multilingual, adding new languages is a piece of cake. You do not have to change the source code at all. Neither do you have to change the resources (forms) in any way. In fact, you do not even have to recompile the application.

Let's add Swedish to the dictionary. Start Language Manager. Choose **File | Open** and browse the `<compiler>\Samples\Tutorial\dcalc.imp`. Choose **Project | Languages**.

The Languages dialog box appears:

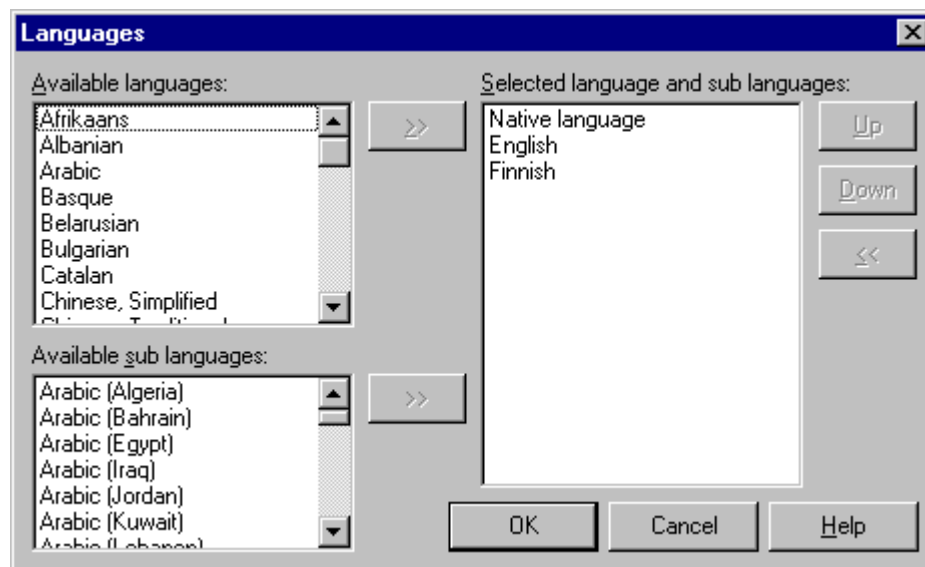


Figure 17. The Languages dialog box lets the user add or remove languages.

Select Swedish from the available languages and press the >> button. Press the OK button to close the dialog box. The Swedish column appears in the dictionary grid. Translate it and save the project. Next time you run Dcalc, Swedish is available.

9

Adding Custom Locales

Normally MULTILIZER gets the locale data (e.g. data and time formatting) from NLSAPI of WIN32. Sometimes the data provided by NLSAPI is not provided or it is missing, however.

For example, you make an application to be used in Lapland (Northern Scandinavia). The language they use is called Sami. NLSAPI does not have any support for Sami (Finland), Sami (Sweden) or Sami (Norway). Also NLSAPI is missing support for Finnish (Sweden). To make these available each dictionary can contain custom locales. MULTILIZER first looks for the locale in the dictionary, if it is not found it looks in NLSAPI.

The system locale table of Language Manager contains all known locales of Windows NT. In addition to these the master locale contains a few custom locales.

Our dictionary contains English and Finnish. NLSAPI supports several English locales such as United States, United Kingdom, Canada, Australia, and New Zealand. Similarly NLSAPI supports one Finnish locale (Finland). The master locale also supports another Finnish locale, Sweden. When Language Manager creates a new dictionary it adds the custom locales to the dictionary. It doesn't add the standard NLSAPI based locales.

Start Language Manager and open the Dcalc project. Choose **Project | Locales**. The Dictionary Locales dialog box appears:

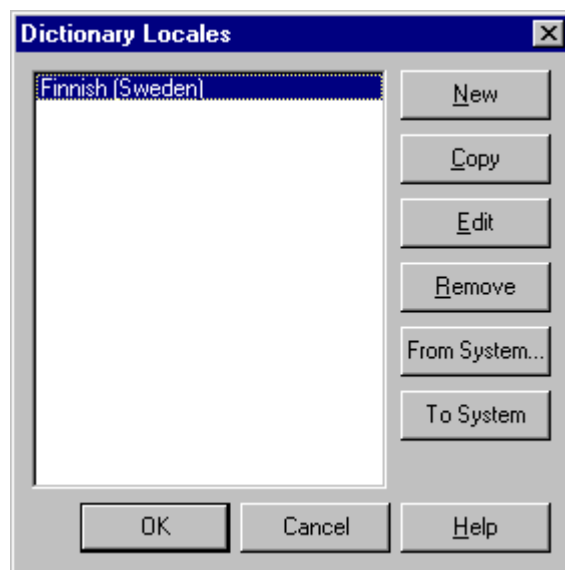


Figure 18. The Dictionary Locales dialog box shows the custom locales of the dictionary..

Our dictionary contains one locale: Finnish (Swedish). This is the only custom locale used by the language of the dictionary. In addition to Finland and Sweden, Finnish is also used in northern Norway. Let's create a new custom locale for Finnish (Norway). We use Finnish (Sweden) as the base. Select Finnish (Sweden). Press the copy button. The New Locale dialog box appears:

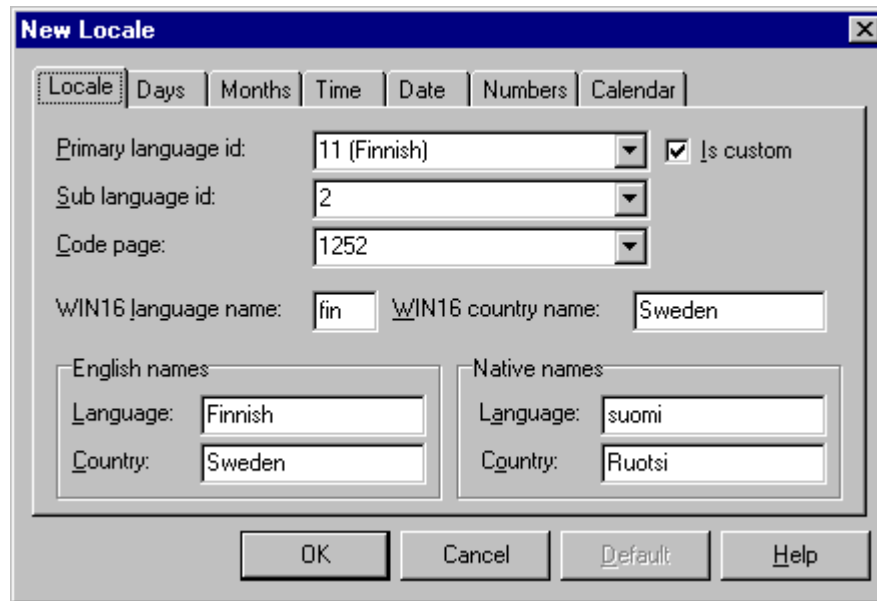


Figure 19. The Locale dialog box is used to enter the locale data.

Change Sub language id to 3 (Norway). Replace Sweden at WIN16 country type and in English names change Country to Norway. Change Ruotsi (Sweden in Finnish) in Native names Country to Norja (Norway in Finnish). Finally press the OK button. Now we have two custom Finnish locales.

Windows NT 4.0 supports eleven English locales. Windows 95 supports nine. Windows 95 lacks the Belize and Trinidad y Tobago locales. If you target to Windows 95 as well and you want to use these locales, add them to the dictionary. Press the **From System** button. The Select Locale dialog box appears. Choose English (Belize) and press **OK**. Repeat this for Trinidad y Tobago.

As a result the Dictionary Locales dialog box should look like this:

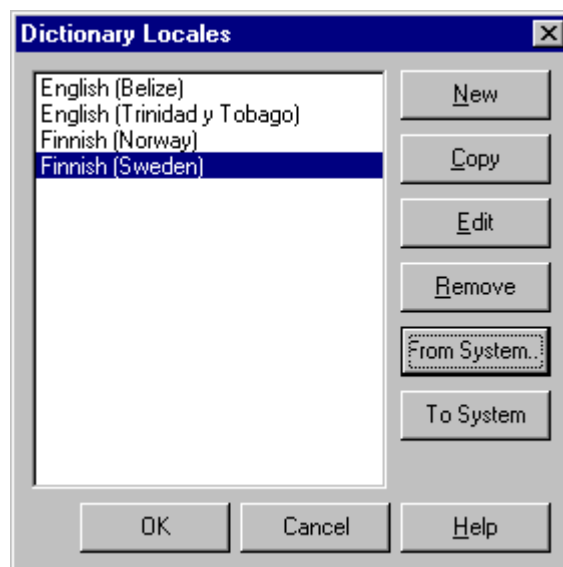


Figure 20. This dictionary contains four custom locales.

Press OK and save the dictionary.

10

Adding Non-Western Languages

VCL applications use Ansi strings. They use single-byte or multi-byte character sets. When rendering the text, Windows uses lookup tables, called code pages, to map the character to a script. . There is a code page for each script. Most Western languages use Western (Latin-1) script that uses code page 1252.

If your dictionary contains only languages using the same code page you can switch back and forth between these languages. For example English, French, German, Finnish and Swedish all use Western script. That's why creating, editing and testing a multilingual application using only Western languages are easy. You can use any Western Windows edition to do the job.

Things get a bit more complicated when you add a language using some other script than Western. For example Polish, Czech and Hungarian use East European script. To properly view East European script you need to have an East European edition of Windows or any Windows NT 4.0.



If you plan to support such languages, we recommend you use Windows NT 4.0. It has many useful features, making it the best environment for developing and testing non-Western languages.

The NT setup CD-ROM contains the LANGPACK directory. This directory contains files to add additional language support for NT. To install these files log on to an account with Administrator privilege, start the Windows NT Explorer and "Right Click" with your mouse on the .INF file named appropriately for the language or group of languages that you wish to add support for. Then select the "Install" option from the menu. Finally reboot your computer.

After reboot your OS can render the scripts of those languages that you selected. Language Manager and all graphical VCL components can now display the non-Western text properly. However, you cannot input non-Western characters. How to do that?

NT can switch the keyboard layout on the fly. This makes it easier to input accented characters. Open the Control Panel. Double click the Keyboard icon. The Keyboard Properties dialog box appears. Select the Input Locales sheet. Press the Add button. The Add Input Locale dialog box appear. Select Polish.

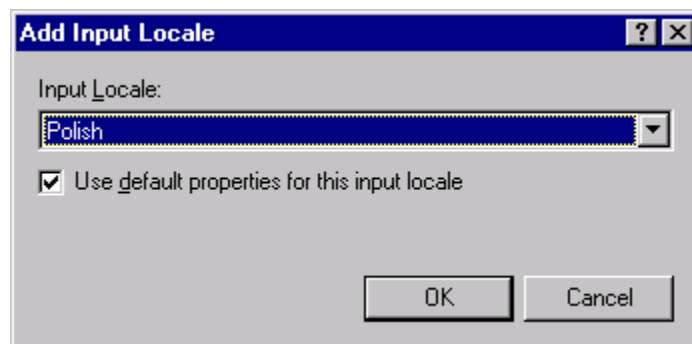


Figure 21. Adding Polish keyboard layout.

Press the OK button to close the dialog box. The Keyboard dialog should look like this:

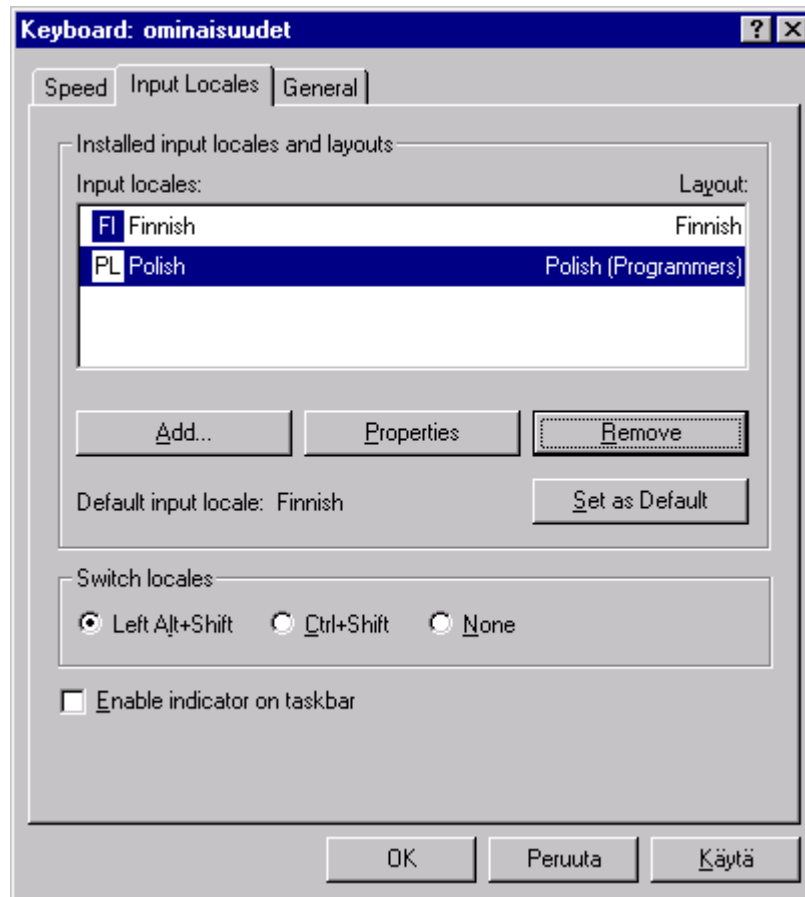


Figure 22. The system contains two input locales: Finnish and Polish.

Press the OK button to close the Keyboard Properties dialog box. This makes Language Manager change to the Polish keyboard layout every time you move to a Polish cell. See more information about Language Manager's online help at Inputting Characters.

If you have a Western NT your default code page is 1252. To test the Polish user interface you have to change the code page to 1251. Open the Control Panel. Double click the Regional Settings icon. The Regional Settings Properties dialog box appears. Select 'Polish' and check 'Set as system default locale' check box.

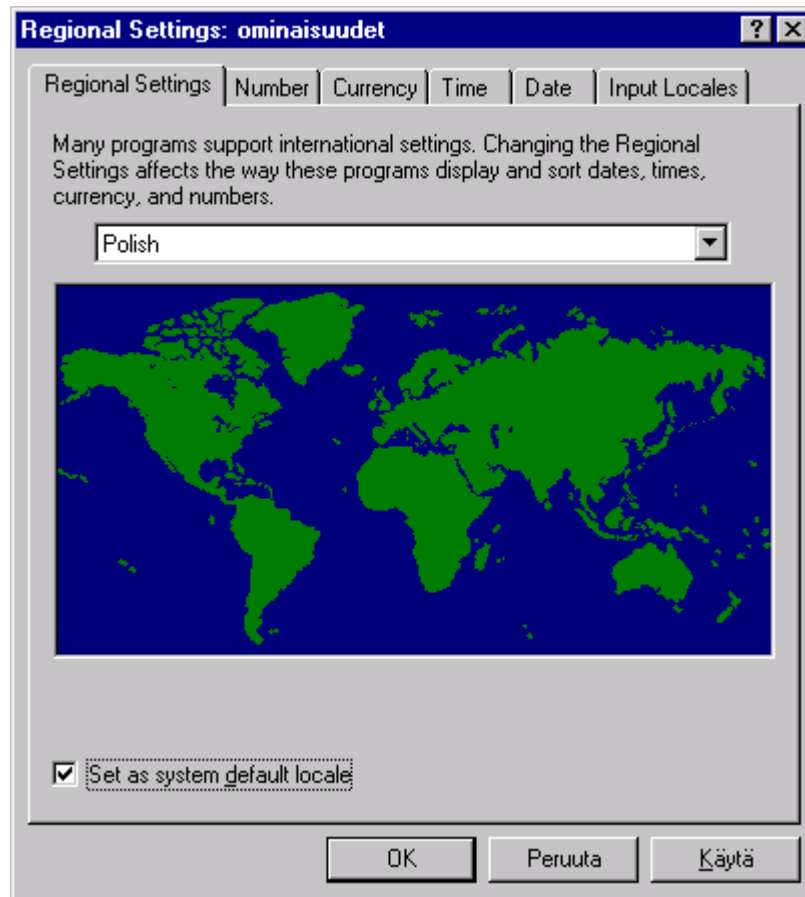


Figure 23. Changing the system locale and code page to Polish.

Press OK. You have to reboot the system. Next time the system restarts, the default code page is 1251. You can now test languages using code page 1251.

11

Adding Far Eastern Languages



Far Eastern (multibyte) languages are available in the professional version of MULTILIZER.

If you use a Far Eastern edition of Windows, adding a Far Eastern language is just like adding any European language. No special steps are required.

If your development system is European, you have to prepare your OS a bit. If you have a European Windows 95 you cannot view or edit Far Eastern languages. Therefore we recommend that you use at least Windows NT 4.0. It has a feature to view far eastern fonts. If you need to have a full Far Eastern support we recommend to use Windows 2000 or Far Eastern Windows versions.

Add the language support for those Far Eastern languages that you plan to use. Keep in mind that you can only edit Far Eastern language using a Far Eastern Windows edition or Windows 2000.

12

Adding Middle Eastern Languages



Middle Eastern (bidirectional) languages are available in the professional version of MULTILIZER.

First of all, you need to have a Windows version supporting bidirectional languages (Arabic or Hebrew Windows, or any Windows 2000) to properly test a multilingual application using a bidirectional language. Adding a European or Far Eastern language did not require any change to your application. This is not the case in the bi-directional language that use right reading order. This means that everything that is normally from left to right is from right to left. To get a proper RLT feeling for the user interface two things must be changed.

Firstly every user item must be mirrored. The picture below contains the left aligned Dcalc using English as the user interface language.

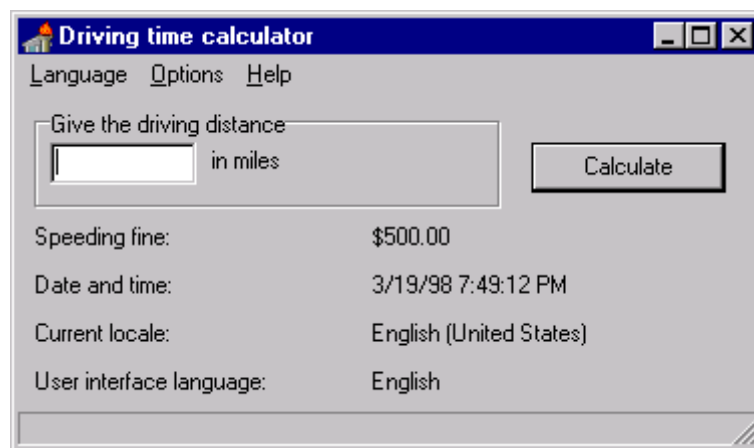


Figure 24. The form using a Western language.

Following picture contains the right aligned Dcalc using Hebrew as the user interface language.

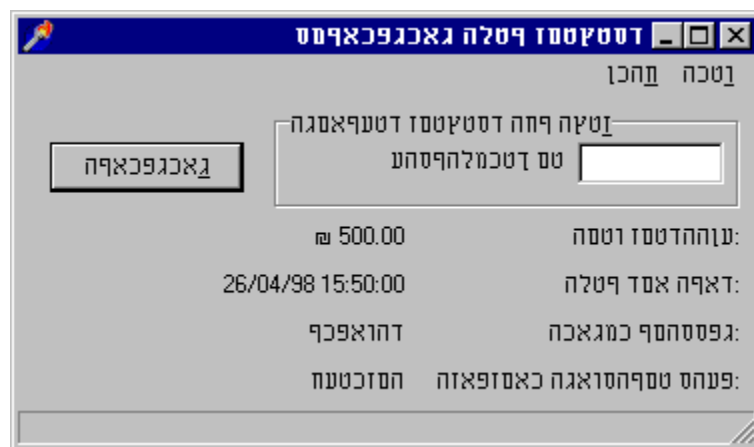


Figure 25. The form using a bidirectional language. The above form does not contain real Hebrew words.



NOTE!

The above image uses the test dictionary. The Hebrew translations are not real Hebrew words.

As you can see, everything is mirrored. MULTILIZER does this automatically.